TP_VBA_Documentation

TP Cours VBA - 27 octobre 2026

Sommaire

1. **Section I : Création de macro avec feuille nouvelle**
2. **Section II : Conteneurs Range et Cells**
3. **Section III : Procédures avec paramètres optionnels**
4. **Section IV : Fonction MAX de deux nombres**
5. **Section V : Fonction factorielle**
6. **Section VI : Gestion d'erreurs en VBA**
7. **Section VII : Recherche de sous-chaîne**
8. **Section VIII : Extraction nom et extension fichier**
9. **Section IX : Saisie et vérification de type de cellule**
10. **Section X : Tableaux 2 dimensions**
11. **Section XI : Tableaux 3 dimensions de réels**
12. **Section XII : Événement Double-clic sur cellule**
13. **Section XIII : Fonction miroir**
14. **Section XIV : Fonction nombre premier**
15. **Section XV : Afficher nom feuille active**
16. **Section XVI : Effacer plage de données avec confirmation**

Section I : Création de macro avec feuille nouvelle

Objectif : Créer une macro qui :

1. Ajoute une nouvelle feuille au classeur ouvert avec un nom unique (NF01, NF02, NF03, ...)
2. Entre des valeurs dans les cellules A1, A2, A3 de la nouvelle feuille
3. Saisit une formule de somme dans la cellule A4
4. Crée des bordures autour du bloc A1:A4
5. Version modifiée demandant à l'utilisateur s'il veut ajouter une nouvelle feuille, avec saisie du nom, et vérification que le nom n'est pas déjà utilisé

Macro de base :

```
Sub CreerFeuilleAvecDonnees()
    Dim ws As Worksheet
    Dim nextSheetNum As Integer
    Dim sheetName As String

    ' Trouver le prochain numéro de feuille unique
    nextSheetNum = 1
    Do While True
        sheetName = "NF" & Format(nextSheetNum, "00")
        On Error Resume Next
        Set ws = ThisWorkbook.Worksheets(sheetName)
        On Error GoTo 0
        If ws Is Nothing Then
            Exit Do ' Nom disponible
        End If
        nextSheetNum = nextSheetNum + 1
    Loop

    ' Ajouter la feuille
    Set ws =
ThisWorkbook.Worksheets.Add(After:=ThisWorkbook.Worksheets(ThisWorkbook.Worksheets.Count))
    ws.Name = sheetName

    ' Entrer des valeurs dans A1, A2, A3
    ws.Range("A1").Value = 10
    ws.Range("A2").Value = 20
    ws.Range("A3").Value = 30

    ' Saisir formule de somme dans A4
    ws.Range("A4").Formula = "=SUM(A1:A3)"

    ' Créer des bordures autour A1:A4
    ws.Range("A1:A4").BorderAround LineStyle:=xlContinuous, Weight:=xlThin

    ' Version modifiée avec saisie utilisateur (voir ci-dessous)
End Sub
```

Version interactive avec validation du nom :

```
Sub CreerFeuilleInteractive()
    Dim sheetName As String
    Dim ws As Worksheet
    Dim sheetExists As Boolean

    ' Demander à l'utilisateur s'il veut ajouter une feuille
```

```
    If MsgBox "Voulez-vous ajouter une nouvelle feuille de calcul?", vbYesNo + vbQuestion = vbNo
Then
        Exit Sub
    End If

    ' Demander le nom de la feuille
    sheetName = InputBox("Entrez le nom de la feuille souhaitée :")

    ' Vérifier si le nom est déjà utilisé
    sheetExists = False
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets(sheetName)
    On Error GoTo 0

    If Not ws Is Nothing Then
        sheetExists = True
        MsgBox "Le nom '" & sheetName & "' est déjà utilisé dans ce classeur." & vbCrLf & _
               "Veuillez choisir un autre nom.", vbExclamation
        Exit Sub
    End If

    ' Si le nom n'existe pas, créer la feuille
    Set ws = ThisWorkbook.Worksheets.Add
    ws.Name = sheetName

    ' Entrer des valeurs par défaut
    ws.Range("A1").Value = 10
    ws.Range("A2").Value = 20
    ws.Range("A3").Value = 30

    ' Ajouter formule de somme
    ws.Range("A4").Formula = "=SUM(A1:A3)"

    ' Ajouter bordures
    ws.Range("A1:A4").BorderAround LineStyle:=xlContinuous, Weight:=xlThin

    MsgBox "Feuille '" & sheetName & "' créée avec succès!", vbInformation
End Sub
```

Section II : Conteneurs Range et Cells

Objectif : Exemples d'utilisation des conteneurs Range et Cells

Règle impérative : Pour des objets comme Range, il faut impérativement utiliser l'instruction `Set` pour leur affecter des valeurs.

1. Donner une valeur à toutes les cellules d'un bloc connu à l'avance

```
Sub RemplirBlocRange()
    ' Remplit la plage A1:B5 avec la valeur 42
    Range("A1:B5").Value = 42
End Sub
```

2. Même chose avec des bornes saisies par l'utilisateur

```
Sub RemplirPlageUtilisateur()
    Dim debutRange As String
    Dim finRange As String
    Dim valeur As String

    debutRange = InputBox("Coordonnée de départ (ex: A1):")
    finRange = InputBox("Coordonnée de fin (ex: B10):")
    valeur = InputBox("Valeur à saisir:")

    ' Utilisation de Range avec variables
    Range(debutRange & ":" & finRange).Value = valeur
End Sub
```

3. Même chose avec la sélection courante et un bouton de commande

```
Sub CreerBoutonSaisie()
    ' Crée un bouton sur la feuille active
    Dim btn As Button
    Set btn = ActiveSheet.Buttons.Add( _
        Left:=ActiveCell.Left, _
        Width:=100, _
        Top:=ActiveCell.Top + ActiveCell.Height, _
        Height:=25)

    btn.Caption = "Saisir une valeur"
    Set btn.OnAction = "SaisirValeurSelection"
End Sub

Sub SaisirValeurSelection()
    Dim valeur As String
```

```
    valeur = InputBox("Valeur à affecter à la sélection actuelle :")
    If valeur <> "" Then
        Selection.Value = valeur
    End If
End Sub
```

Section III : Procédures avec paramètres optionnels

Objectif : Écrire des procédures avec paramètres facultatifs

1. Procédure tracecercle avec rayon optional

```
Sub tracecercle(ByVal x As Double, ByVal y As Double, Optional ByVal r As Double = 1)
    ' Trace un cercle de centre (x, y) et de rayon r
    Dim i As Integer
    Dim angle As Double
    Dim points() As Point

    ReDim points(0 To 360)

    For i = 0 To 360
        angle = i * Application.Pi / 180
        points(i).X = x + r * Cos(angle)
        points(i).Y = y + r * Sin(angle)
    Next i

    ' Affichage des points (implémentation dépendante)
    MsgBox "Cercle tracé de centre (" & x & ", " & y & ") et rayon " & r
End Sub
```

Appels à la procédure :

```
' Avec paramètres ordonnés
Call tracecercle(0, 0, 5)

' Avec paramètres nommés
Call tracecercle(x:=0, y:=0, r:=5)

' Avec seul le rayon en nommé
Call tracecercle(r:=3)
```

2. Fonction produit de 5 variables avec paramètres optionnels

```
Function ProduitCinqVars( _
    ByVal a As Double, _
    ByVal b As Double, _
    ByVal c As Double, _
    Optional ByVal d As Double = 1, _
    Optional ByVal e As Double = 2) As Double

    ProduitCinqVars = a * b * c * d * e
End Function
```

Exemples d'appels :

```
' Tous les paramètres
Debug.Print ProduitCinqVars(1, 2, 3, 4, 5) ' = 120

' Avec les défauts
Debug.Print ProduitCinqVars(1, 2, 3) ' = 1*2*3*1*2 = 12

' Paramètres nommés
Debug.Print ProduitCinqVars(a:=1, b:=2, e:=3) ' = 1*2*1*1*3 = 6
```

Section IV : Fonction MAX de deux nombres

Objectif : Fonction qui prend deux réels et renvoie le plus grand

```
Function MaxDeuxNombres(ByVal x As Double, ByVal y As Double) As Double
    If x >= y Then
        MaxDeuxNombres = x
    Else
        MaxDeuxNombres = y
    End If
End Sub
```

Procédure de test :

```
Sub TestMaxDeuxNombres()
    Dim val1 As Double
    Dim val2 As Double
```

```
    Dim resultat As Double

    val1 = InputBox("Entrez la première valeur :")
    val2 = InputBox("Entrez la deuxième valeur :")

    resultat = MaxDeuxNombres(val1, val2)
    MsgBox "Le plus grand des deux nombres est : " & resultat, vbInformation
End Sub
```

Section V : Fonction factorielle

Objectif : Deux versions d'une fonction factorielle (boucle for et récursive), avec validation de saisie

1. Version itérative (boucle For)

```
Function FactorielleIterative(ByVal n As Long) As Double
    Dim i As Long
    Dim resultat As Double

    resultat = 1
    For i = 2 To n
        resultat = resultat * i
    Next i

    FactorielleIterative = resultat
End Function
```

2. Version récursive

```
Function FactorielleRecursive(ByVal n As Long) As Double
    If n <= 1 Then
        FactorielleRecursive = 1
    Else
        FactorielleRecursive = n * FactorielleRecursive(n - 1)
    End If
End Function
```

3. Procédure de test avec validation

```
Sub TestFactorielle()
    Dim n As Long
    Dim resultat As Double

    ' Demande de saisie tant que la valeur est invalide
    Do
        n = InputBox("Entrez la valeur de n pour calculer sa factorielle :", "Saisie de n")

        ' Vérifier si c'est un nombre entier positif ou nul
        If Not IsNumeric(n) Then
            MsgBox "Veuillez entrer une valeur numérique.", vbExclamation
            Continue Do
        End If

        n = CInt(n)

        ' Vérifier si négatif
        If n < 0 Then
            MsgBox "La factorielle d'un nombre négatif n'a pas de sens. Veuillez réessayez.", vbExclamation
            Continue Do
        End If

        ' Condition d'arrêt si n > 10
        If n > 10 Then
            MsgBox "Attention : n = " & n & " est trop grand (> 10). Sortie de la procédure.",
vbExclamation
            Exit Sub
        End If

        ' Sortir de la boucle si tout est valide
        Exit Do

    Loop

    ' Calculer la factorielle des deux façons
    resultat = FactorielleIterative(n)
    MsgBox "Factorielle(" & n & ") (itérative) : " & resultat, vbInformation

    resultat = FactorielleRecursive(n)
    MsgBox "Factorielle(" & n & ") (récursive) : " & resultat, vbInformation
End Sub
```

Précaution : Par convention : 0! = 1 et 1! = 1, et n! = n*(n-1)*(n-2)...2*1

Section VI : Gestion d'erreurs en VBA

Objectif : Comment on gère les erreurs en VBA (3 manières) et exemples

3 méthodes de gestion d'erreurs :

1. **Aucune gestion (programme plante)** - MAUVAISE SOLUTION
2. **Option Resume** - Reprend à l'instruction suivante
3. **Option ErrorHandler** - Gestion structurée avec Exit Sub

Exemple sans gestion d'erreur (Mauvaise solution) :

```
Sub essai2_sans_gestion_derreur()
    ' Sans gestion d'erreur,
    ' à la survenance d'une erreur le programme plante directement
    ' impossible de poursuivre l'exécution des instructions qui suivent

    Dim t(10) As Double
    t(11) = 25.321 ' ATTENTION instruction qui génère une erreur
    MsgBox "Nous sommes passés"
    Exit Sub
End Sub
```

2. Gestion avec seulement Resume

```
Sub EssaiResume()
    On Error Resume Next

    Dim t(10) As Double
    t(11) = 25.321 ' Cette ligne génère une erreur

    ' Reprend ici après l'erreur
    If Err.Number <> 0 Then
        MsgBox "Erreur n°" & Err.Number & ": " & Err.Description
        Err.Clear
    End If

    MsgBox "Nous sommes passés après l'erreur"
End Sub
```

Que se passe-t-il ? L'instruction `Resume` repartir de l'instruction qui a causé l'erreur, ce qui peut provoquer une boucle infinie si l'erreur n'est pas corrigée. C'est une solution limitée.

3. Gestion complète avec Error Handler

```
Sub EssaiGestionErreurComplete()
    On Error GoTo GestionErreur

    Dim t(10) As Double
    t(11) = 25.321 ' Cette ligne génère une erreur

    ' Ces instructions ne s'exécuteront pas en cas d'erreur
    MsgBox "Nous sommes passés avant l'erreur"
    Exit Sub

GestionErreur:
    MsgBox "Erreur n°" & Err.Number & ": " & Err.Description & vbCrLf & _
           "Impossible de continuer.", vbCritical

    ' On peut choisir de continuer ou d'arrêter
    Resume Next
End Sub
```

Section VII : Recherche de sous-chaîne dans une chaîne

Objectif : Fonction qui recherche une sous-chaîne à partir d'une position de départ

```
Function RechercheSousChaine( _
    ByVal chainePrincipale As String, _
    ByVal sousChaine As String, _
    Optional ByVal positionDebut As Long = 1) As Long

    ' Recherche la sous-chaine à partir de la position donnée
    Dim pos As Long

    pos = InStr(positionDebut, chainePrincipale, sousChaine)

    If pos > 0 Then
        RechercheSousChaine = pos ' Position trouvée (1-indexed)
    Else
        RechercheSousChaine = 0 ' Valeur impossible pour montrer que non trouvé
    End If
End Function
```

Procédure de test :

```vba
Sub TestRechercheSousChaine()
    Dim chaine As String
    Dim sousChaine As String
    Dim posDebut As Long
    Dim position As Long

    chaine = InputBox("Entrez la chaîne principale :")
    sousChaine = InputBox("Entrez la sous-chaine à rechercher :")
    posDebut = InputBox("Entrez la position de départ (1 par défaut):")
    If posDebut = "" Then posDebut = 1

    position = RechercheSousChaine(chaine, sousChaine, posDebut)

    If position > 0 Then
        MsgBox "La sous-chaine est trouvée à la position " & position, vbInformation
    Else
        MsgBox "La sous-chaine n'est pas présente dans la chaîne donnée à partir de la position " &
posDebut, vbExclamation
    End If
End Sub
```

Section VIII : Extraction nom et extension fichier

Objectif : Fonction qui extrait le nom et l'extension d'un nom de fichier

```vba
Function ExtraireNomExtension(ByVal nomFichier As String) As String
    ' Extrait le nom et l'extension d'un nom de fichier
    ' Exemple : "classeur.xls" -> "classeur et xls"

    Dim pointPosition As Long

    ' Trouver la position du dernier point
    pointPosition = InStrRev(nomFichier, ".")

    If pointPosition > 0 And pointPosition < Len(nomFichier) Then
        ' Extraire le nom (avant le point)
        Dim nom As String
        nom = Left(nomFichier, pointPosition - 1)

        ' Extraire l'extension (après le point)
        Dim extension As String
        extension = Mid(nomFichier, pointPosition + 1)
```

```
        ExtraireNomExtension = nom & " et " & extension
    Else
        ' Pas de point trouvé
        ExtraireNomExtension = nomFichier & " (pas d'extension détectée)"
    End If
End Function
```

Exemples :

```
Debug.Print ExtraireNomExtension("classeur.xls")     ' Affiche : "classeur et xls"
Debug.Print ExtraireNomExtension("document.pdf")     ' Affiche : "document et pdf"
Debug.Print ExtraireNomExtension("archive.zip")      ' Affiche : "archive et zip"
Debug.Print ExtraireNomExtension("sans_extension")   ' Affiche : "sans_extension (pas d'extension
détectée)"
```

Section IX : Saisie et vérification de type de cellule

Objectif : Procédure qui saisit la valeur d'une cellule et vérifie si c'est une date ou un nombre

```
Sub VerifierTypeCellule()
    Dim valeurSaisie As String
    Dim valeur As Variant
    Dim choix As VbMsgBoxResult

    ' Saisir la valeur d'une cellule
    valeurSaisie = InputBox("Entrez une valeur de cellule à vérifier :")

    ' Essayer de la convertir
    On Error Resume Next
    valeur = valeurSaisie
    On Error GoTo 0

    ' Menu de choix
    choix = MsgBox "Voulez-vous vérifier si le type est :" & vbCrLf & _
                vbCrLf & "1. Une date" & vbCrLf & _
                "2. Un entier", _
                vbQuestion + vbYesNoNo, "Type de données"

    Select Case choix
        Case vbYes
            ' Vérifier si c'est une date
```

```
            If IsDate(valeurSaisie) Then
                MsgBox "C'est une date!" & vbCrLf & _
                    "Formats possibles :" & vbCrLf & _
                    "- Format court : " & Format(CDate(valeurSaisie), "dd/mm/yyyy") & vbCrLf & _
                    "- Format long : " & Format(CDate(valeurSaisie), "dddd dd mmmm yyyy") & vbCrLf & _
                    "- Format US : " & Format(CDate(valeurSaisie), "mm/dd/yyyy")
            Else
                MsgBox "Ce n'est pas une date.", vbInformation
            End If

        Case vbNo
            ' Vérifier si c'est un entier
            If IsNumeric(valeurSaisie) Then
                Dim nb As Double
                nb = CDouble(valeurSaisie)

                If nb <= 100 Then
                    MsgBox "C'est un entier <= 100." & vbCrLf & _
                        "Affichage en pourcentage : " & Format(nb / 100, "0%")
                Else
                    MsgBox "C'est un entier > 100." & vbCrLf & _
                        "Affichage avec symbole € : " & nb & " €"
                End If
            Else
                MsgBox "Ce n'est pas un nombre valide.", vbInformation
            End If

        Case vbNull
            ' Aucun bouton pressé
            MsgBox "Aucune vérification effectuée.", vbInformation
    End Select
End Sub
```

Section X : Tableaux 2 dimensions

Objectif : Créer un tableau d'entiers en 2 dimensions 5 x 6, saisir les valeurs par l'utilisateur et les afficher

```
Sub Tableau2Dimensions()
    Dim tab(1 To 5, 1 To 6) As Integer
    Dim i As Integer, j As Integer

    ' Saisir les valeurs par l'utilisateur
```

```
    For i = 1 To 5
        For j = 1 To 6
            tab(i, j) = InputBox("Entrez la valeur pour tab(" & i & ", " & j & "):")
        Next j
    Next i

    ' Afficher les valeurs en précisant leur indice
    Dim message As String
    message = "Contenu du tableau 5x6 : " & vbCrLf & vbCrLf

    For i = 1 To 5
        For j = 1 To 6
            message = message & "tab(" & i & ", " & j & ") = " & tab(i, j) & vbTab
        Next j
        message = message & vbCrLf
    Next i

    MsgBox message, vbInformation, "Tableau 2D"
End Sub
```

Section XI : Tableaux 3 dimensions de réels

Objectif : Créer un tableau de réels (à virgule) en 3 dimensions 5 x 4 x 3, saisir les valeurs et les afficher

```
Sub Tableau3Dimensions()
    Dim tab(1 To 5, 1 To 4, 1 To 3) As Double
    Dim i As Integer, j As Integer, k As Integer

    ' Saisir les valeurs par l'utilisateur
    For i = 1 To 5
        For j = 1 To 4
            For k = 1 To 3
                tab(i, j, k) = InputBox("Entrez la valeur pour tab(" & i & ", " & j & ", " & k & "):")
            Next k
        Next j
    Next i

    ' Afficher les valeurs en précisant leur indice
    Dim message As String
    message = "Contenu du tableau 5x4x3 : " & vbCrLf & vbCrLf

    For i = 1 To 5
```

```
        For j = 1 To 4
            For k = 1 To 3
                message = message & "tab(" & i & ", " & j & ", " & k & ") = " & tab(i, j, k) & vbTab
            Next k
            message = message & vbCrLf
        Next j
        message = message & vbCrLf & "---" & vbCrLf
    Next i

    MsgBox message, vbInformation, "Tableau 3D"
End Sub
```

Section XII : Événement Double-clic sur cellule

Objectif : Procédure événementielle qui, au double-clic sur une cellule, affiche un message avec l'adresse et le nom de la feuille, et positionne Cancel = True

Cette procédure doit être placée dans le classeur ou la feuille spécifique dans l'éditeur VBA (événement `Worksheet_BeforeDoubleClick` ou `Workbook_SheetBeforeDoubleClick`).

Pour une feuille spécifique :

```
Private Sub Worksheet_BeforeDoubleClick(ByVal Target As Range, Cancel As Boolean)
    ' Afficher message avec adresse et nom de feuille
    MsgBox "Double-clic sur la cellule " & Target.Address & vbCrLf & _
           "Dans la feuille : " & Me.Name, vbInformation

    ' Empêcher la cellule de passer en mode mise à jour
    Cancel = True
End Sub
```

Pour tout le classeur (Workbook) :

```
Private Sub Workbook_SheetBeforeDoubleClick(ByVal Sh As Object, ByVal Target As Range, Cancel As Boolean)
    MsgBox "Double-clic sur la cellule " & Target.Address & " dans la feuille " & Sh.Name, vbInformation
    Cancel = True
End Sub
```

Note : Pour que cela fonctionne, il faut :

1. Ouvrir l'éditeur VBA (Alt+F11)
2. Dans l'Explorateur de projet, double-cliquer sur la feuille concernée
3. Coller le code dans l'événement `Worksheet_BeforeDoubleClick`
4. Les événements Worksheet doivent avoir `Application.EnableCancelKey = xlErrorHandler` ou la gestion doit être activée

---

Section XIII : Fonction miroir

Objectif : Fonction qui renvoie une chaîne après avoir inversé les caractères

```
Function Miroire(ByVal chaine As String) As String
    Dim i As Integer
    Dim resultats As String

    ' Inverser les caractères de la chaîne
    resultats = ""
    For i = Len(chaine) To 1 Step -1
        resultats = resultats & Mid(chaine, i, 1)
    Next i

    Miroire = resultats
End Function
```

Exemples :

```
Debug.Print Miroire("Bonjour")     ' Affiche : "ruojnoB"
Debug.Print Miroire("VBA")         ' Affiche : "ABV"
Debug.Print Miroire("")            ' Affiche : ""
Debug.Print Miroire("12345")       ' Affiche : "54321"
```

Section XIV : Fonction nombre premier

Objectif : Fonction vérifiant si un nombre est premier

```
Function EstPremier(ByVal n As Long) As Boolean
    Dim i As Long
    Dim racine As Double
```

```
    ' 0, 1 ne sont pas premiers
    If n <= 1 Then
        EstPremier = False
        Exit Function
    End If

    ' 2 est premier
    If n = 2 Then
        EstPremier = True
        Exit Function
    End If

    ' Éliminer les nombres pairs
    If n Mod 2 = 0 Then
        EstPremier = False
        Exit Function
    End If

    ' Vérifier les diviseurs impairs jusqu'à la racine carrée
    racine = Sqr(n)
    For i = 3 To racine Step 2
        If n Mod i = 0 Then
            EstPremier = False
            Exit Function
        End If
    Next i

    EstPremier = True
End Function
```

Exemples :

```
Debug.Print EstPremier(2)    ' Vrai
Debug.Print EstPremier(3)    ' Vrai
Debug.Print EstPremier(4)    ' Faux
Debug.Print EstPremier(5)    ' Vrai
Debug.Print EstPremier(97)   ' Vrai
Debug.Print EstPremier(100)  ' Faux
Debug.Print EstPremier(0)    ' Faux
Debug.Print EstPremier(1)    ' Faux
```

Section XV : Afficher nom feuille active

Objectif : Procédure qui affiche le nom de la feuille de calcul active, associée à un bouton de commande

Procédure :

```
Sub AfficherNomFeuilleActive()
    MsgBox "Feuille de calcul active : " & ActiveSheet.Name
End Sub
```

Création du bouton :

1. Dans Excel, allez sur l'onglet "Développeur"
2. Cliquez sur "Insérer" > "Bouton (contrôle ActiveX)" ou "Bouton d'appel de macro"
3. Dessinez le bouton sur la feuille
4. Dans la boîte de dialogue "Assigner une macro", sélectionner `AfficherNomFeuilleActive`
5. Cliquez sur OK

Ou en VBA :

```
Sub CreerBoutonAfficherFeuille()
    Dim btn As Button
    Set btn = ActiveSheet.Buttons.Add( _
        Left:=50, _
        Width:=100, _
        Top:=50, _
        Height:=30)

    btn.Caption = "Feuille active"
    btn.OnAction = "AfficherNomFeuilleActive"
End Sub
```

Section XVI : Effacer plage de données avec confirmation

Objectif : Procédure qui efface les valeurs d'une plage de données dont les bornes sont saisies par l'utilisateur, avec demande de confirmation et bouton par défaut sur Non

```
Sub EffacerPlageAvecConfirmation()
    Dim debutRange As String
    Dim finRange As String
    Dim reponse As VbMsgBoxResult
```

```
    ' Saisir les bornes de la plage
    debutRange = InputBox("Coordonnée de départ de la plage (ex: A1):")
    finRange = InputBox("Coordonnée de fin de la plage (ex: B10):")

    ' Demander confirmation avec bouton par défaut sur Non
    reponse = MsgBox "Voulez-vous vraiment effacer les données de la plage " & _
                     debutRange & " : " & finRange & " ?", _
                     vbYesNo + vbDefaultButton2, "Confirmation"

    ' Si l'utilisateur clique sur Oui
    If reponse = vbYes Then
        ' Effacer la plage
        On Error Resume Next
        Range(debutRange & ":" & finRange).ClearContents
        On Error GoTo 0

        MsgBox "Les données de la plage " & debutRange & " : " & finRange & " ont été effacées.",
vbInformation
    Else
        MsgBox "Opération annulée. Aucune donnée n'a été effacée.", vbInformation
    End If
End Sub
```

TP Assistés

VBA L'essentiel (pages de référence)

- **P 214 : Courbe de Gauss** - Génération de points suivant une distribution normale
- **P 254 : Créer formulaires** - Création de user forms interactifs
- **26 questions VBA** - Questions de révision sur les concepts abordés

Exercices du manuel "Apprenez le langage VBA"

- **Exo p 36 -> 56** - Exercices fondamentaux
- **Exo p 57 -> 76 Formules** - Travail avec les formules Excel
- **Exo p 77 -> 131 Formule de gestion des ventes** - Application complète de gestion commerciale
- **Exo p 132 -> 151 Protection du classeur** - Sécurisation des classeurs et feuilles

Autres ressources BDI

- **BDI p 96** :
  1. Gestion Foot (football)
  2. Gestion Association
  3. Facturation
- **QCM p 176** - Questions à choix multiples sur VBA

Guide de création du fichier Excel avec tests

Le fichier Excel suivant doit contenir une feuille par section du TP, avec une macro de test pour chaque concept. Voici la structure recommandée :

Structure du classeur `Tests_VBA.xlsm` :

1. **Feuille "Accueil"** - Boutons pour accéder à chaque section de tests
2. **Feuille "Section_I"** - Tests de création de macro avec feuille
3. **Feuille "Section_II"** - Tests Range/Cells
4. **...** (continue pour chaque section)
5. **Feuille "Section_XVI"** - Tests effacement plage avec confirmation

Exemple de structure pour une feuille de test :

```
' Exemple : Bouton de test Section I
Sub Test_CreerFeuille()
    Call CreerFeuilleAvecDonnees
    MsgBox "Test Section I : Création de feuille réussi!", vbInformation
End Sub

Sub Test_CreerFeuilleInteractive()
    Call CreerFeuilleInteractive
    MsgBox "Test Section I : Version interactive réussi!", vbInformation
End Sub
```